Abstraction

Wednesday, October 15, 2025 3:40 PM

Definition:

'''Abstraction is the process of **hiding the internal implementation details** and showing only the **necessary functionalities** to the user.'''

*It focuses on **what to do**, and hides **how it is done**.

Real-Life Example

Think of an **ATM Machine**:

- When you withdraw cash, you only interact with **buttons and screen**.
- You don't know **how** the machine connects to the server, **verifies** your balance, or **dispenses** money.

You see only the functionality, not the implementation.

That's exactly what **abstraction** does in programming.

Purpose of Abstraction

1. To provide **security** by hiding complex details.
2. To make the system **modular and maintainable**.
3. To focus on **essential operations** only.
4. To achieve **code reusability** and **flexibility**.

Key Components of Abstraction

Abstract Method

- A method that **contains only declaration** but **no definition (implementation)**.
- Defined using the @abstractmethod decorator from the abc module.
- Must be **overridden** in the subclass.

Syntax:

```
from abc import ABC, abstractmethod
class ClassName(ABC):        # Abstract class
    @abstractmethod
    def method_name(self):
        pass                 # No body (declaration only)
```

Abstract Class

- A class that **contains at least one abstract method**.
- We **cannot create an object** of an abstract class.
- It acts as a **blueprint** for other (child) classes.

Syntax:

```
from abc import ABC, abstractmethod
class Parent(ABC):   # Abstract Class
    @abstractmethod
    def func(self):
        pass
class Child(Parent):  # Concrete Class
    def func(self):
        print("Function implemented")
obj = Child()
obj.func()
```

Concrete Class

- A class that **implements all the abstract methods** of its parent abstract class.
- We **can create objects** for concrete classes.

Syntax:

```
class ConcreteClass(AbstractClass):
    def abstract_method(self):
        print("Implementation provided here")
```

Note:

Inheritance is a pre-requisite for abstraction

Because abstract methods defined in the parent class are **implemented** in child classes.

ATM Example (Without Abstraction)

```
class ATM:
    def withdraw(self, amount):
        print(f'Withdrawing {amount} from account')
        print('Connecting to bank server...')
        print('Checking balance...')
        print('Debiting account...')
        print('Dispensing cash...')
```

```
atm = ATM()
atm.withdraw(2000)
```

Drawback:

All implementation details are **visible** to the user.

ATM Example (With Abstraction)

```
from abc import ABC, abstractmethod
class ATM(ABC):
    @abstractmethod
    def withdraw(self, amount):
        pass
class HDFC_ATM(ATM):
    def withdraw(self, amount):
        print("Connecting to HDFC bank server...")
        print(f"Withdrawing ₹{amount}")
        print("Please take your cash.")
atm = HDFC_ATM()
atm.withdraw(2000)
```

Output:

```
Connecting to HDFC bank server...
Withdrawing ₹2000
Please take your cash.
```

Explanation:

- ATM → Abstract class (defines structure)
- HDFC_ATM → Concrete class (implements behavior)
- Implementation details are **hidden** from the user.

Trainer Example

```
from abc import ABC, abstractmethod
class Trainer(ABC):
    @abstractmethod
    def do_Assignment(self):
        pass
class GoodStudent(Trainer):
    def do_Assignment(self):
        print("Homework Done ")
```

```
class BadStudent(Trainer):
    pass  # Forgot to do homework
# t = Trainer() # Error - abstract class cannot be instantiated
good = GoodStudent()
good.do_Assignment()
```

Output:

```
Homework Done
```

Advanced Example – ATM with Multiple Banks

```
from abc import ABC, abstractmethod
class ATM(ABC):
    def __init__(self, bank_name):
        self.bank_name = bank_name
    @abstractmethod
    def connect_to_server(self):
        pass
    @abstractmethod
    def withdraw(self, amount):
        pass
class HDFC_ATM(ATM):
    def connect_to_server(self):
        print(f'Connecting to {self.bank_name} server...')
def withdraw(self, amount):
        self.connect_to_server()
        print(f'Withdrawing ₹{amount} from HDFC account...')
        print('Transaction successful!')
atm = HDFC_ATM('HDFC')
atm.withdraw(2000)
```

Output:

```
Connecting to HDFC server...
Withdrawing ₹2000 from HDFC account...
Transaction successful!
```

Calculator Example (Multiple Abstract Methods)

```
from abc import ABC, abstractmethod
class Calculator(ABC):
   @abstractmethod
   def add(self, a, b):
         pass
   @abstractmethod
   def sub(self, a, b):
         pass
   @abstractmethod
   def mul(self, a, b):
         pass
   @abstractmethod
   def div(self, a, b):
         pass

class Casio(Calculator):
   def add(self, a, b):
          print("Sum:", a + b)

   def sub(self, a, b):
         print("Sub:", a - b)

   def mul(self, a, b):
         print("Mul:", a * b)

   def div(self, a, b):
         print("Div:", a / b)
u1 = Casio()
u1.add(10, 20)
u1.sub(20, 10)
u1.mul(5, 6)
u1.div(10, 2)
```

Output:

```
Sum: 30
Sub: 10
Mul: 30
Div: 5.0
```

Hotel Example

```
from abc import ABC, abstractmethod
class Hotel(ABC):
   @abstractmethod
   def cook(self):
      pass
class VegBiryani(Hotel):
   def cook(self):
     Implementation="Cooking: Chop veggies → Boil rice → Add masala → Serve
     hot "
      print('Have Your food')
c1 = VegBiryani()
c1.cook()
```

Search Engine Example

```
from abc import ABC, abstractmethod
class Search(ABC):
   @abstractmethod
   def searching(self, query):
      pass
class Google(Search):
   def searching(self, query):
      print(f'Showing top results for "{query}" on Google.')
addict1 = Google()
addict1.searching("Python OOP concepts")
```

Output:

Showing top results for "Python OOP concepts" on Google.

Summary Table

Concept	Description	Object Creation
Abstraction	Hiding implementation details and showing only functionality	In subclass only
Abstract Method	Method with declaration but no definition	Cannot execute directly
Abstract Class	Contains ≥1 abstract method	Cannot create object

Concrete Class	Implements all abstract methods	Object creation allowed
Module Required	from abc import ABC, abstractmethod	-

Advantages of Abstraction

1. Provides **security** (implementation hidden)
2. Increases **code reusability**
3. Makes code **modular** and **easy to maintain**
4. Allows **standardization** of functionalities across subclasses
5. Promotes **loose coupling** (less dependency between classes)